

01 Project crux

Read top to bottom: the exact path from an entered record to a result.

STAGE	WHAT ACTUALLY HAPPENS
User / CSV	Data page accepts mapped CSV columns or manual form values; Players must exist first.
Validation	Required columns/cells, dates, types, ranges, relationships and player/date uniqueness are checked.
SQLite database	SQLAlchemy stores accepted rows in eight tables. Invalid rows are not stored.
Retrieval	Services read table rows into Pandas frames, grouped by player and ordered by date where needed.
Feature engineering	Earlier records become calendar-window sums, means, slopes, counts and latest values.
ML / analytics	Three supervised pipelines train on labeled outcomes; trends calculate statistics; K-Means groups profiles.
Prediction / evaluation	Saved models predict probabilities or a rating. Held-out tests produce metrics; clustering produces profiles.
Streamlit UI	Dashboard, Data, Players, ML Predictions, Health and Model Analysis display results.

MASTER PATH

RAW DATA > VALIDATION > SQLITE > RETRIEVAL > FEATURES > MODEL OR ANALYTICS > OUTPUT > UI

Six major capabilities

MODULE	INPUT TO FINAL OUTPUT
Injury prediction	Training + recovery + injuries + profile > 7-day onset probability and Low/Medium/High.
Performance prediction	Prior matches + training + recovery + profile > next-match rating estimate.
Health monitoring	Health + medical tests + health_events > 7-day event probability and risk band.
Health trends	Observed health/test series > rolling charts, latest change and trend direction.
Health clustering	Recent health + available labs > K-Means group, profile and PCA chart.
Model analysis	Training readiness + validation comparison + held-out test + global feature importance.

02 Technologies and roles

Only packages and systems present in the current repository.

TECHNOLOGY	PURPOSE	WHERE USED
Python 3.12	Application language, feature calculations and ML orchestration.	Dockerfile; all .py files
Streamlit	Page navigation, forms, buttons, tables, metrics and charts.	app.py; views/
SQLite	Persistent default database football_ml.db.	database/database.py
SQLAlchemy + SQL	ORM models, constraints, select/count queries and transactions.	database/; services/data_service.py
Pandas	Read CSV, create frames, filter histories, calculate sums/means.	views/data.py; services/
NumPy	Missing values, arrays, slopes, standard deviation, RMSE.	services/; ml/train.py
Scikit-learn	Imputation, scaling, one-hot coding, models, split metrics, K-Means, PCA.	ml/; services/health_service.py
Plotly	Confusion matrix, feature importance and clustering scatter plots.	views/health.py; views/model_analysis.py
joblib	Serialize and reload fitted model pipelines.	ml/train.py; ml/health.py
Docker	Builds Python 3.12 image and runs Streamlit on port 8501.	Dockerfile
pytest	Project test suite.	tests/; pytest.ini
python-dotenv	Declared dependency; no import found in application files.	requirements.txt

Storage is separate from trained model files

SQLite holds imported data. A trained pipeline is a joblib file plus JSON metadata under data/model_artifacts; Health uses its health subfolder. A dataset fingerprint and feature version guard artifact reuse. The database URL can be overridden with FOOTBALL_ML_DATABASE_URL.

WHAT IS ABSENT

No XGBoost or Matplotlib dependency or import appears in the current application. No XGBoost model is trained.

03 Raw data: exact CSV contract

R = required; O = optional. Types: S text, N numeric, I integer, D ISO date, B Boolean.

FILE > SQLITE TABLE	COLUMNS (type; role)
players.csv > players	R player_id S key, name S display, age I years, position S role; O height N cm, weight N kg.
training.csv > training	R player_id S, date D, duration_min N minutes, rpe N, distance N km, sprint_distance N m, high_speed_distance N m.
recovery.csv > recovery	R player_id S, date D, sleep_hours N, sleep_quality N, hrv N, resting_hr N, soreness N, stress N.
injuries.csv > injuries	R player_id S, injury_date D, injury_occurred B target log; O injury_type S, days_missed I.
matches.csv > matches	R player_id S, match_date D, minutes_played I, goals I, assists I, shots I, passes_completed I, key_passes I, rating N target.
health.csv > health_records	R player_id S, date D, resting_hr N, hrv N, sleep_hours N, sleep_quality N, soreness N, fatigue N, stress N, energy_level N, hydration_status N; O weight N, body_fat_pct N, blood_pressure_sys N, blood_pressure_dia N, oxygen_saturation N, body_temperature N.
medical_tests.csv > medical_tests	R player_id S, test_date D; O hemoglobin, hematocrit, wbc_count, platelet_count, ferritin, serum_iron, vitamin_d, vitamin_b12, glucose, creatinine, crp (all N).
health_events.csv > health_events	R player_id S, event_date D, health_event B target log; 1 = documented event, 0 = confirmed event-free day.

Four different things

RAW INPUT = one of the CSV columns above. TARGET = injury_occurred over a future 7-day window, a match rating on its match date, or health_event over a future 7-day window. DERIVED FEATURE = a value computed only from earlier records. PREDICTION = model output on a selected date; it is never imported as an input.

What each field family means and who consumes it

DATASET	FIELD MEANING / USED BY
players	ID links every record; name is display text. Age and position enter Injury/Performance ML; height and profile weight are stored/displayed, not ML features.
training	Date anchors the session; duration and RPE form workload; distance checks sprint/high-speed consistency; sprint and high-speed distances form seven-day loads. Injury/Performance.
recovery	Sleep duration/quality, HRV, resting HR, soreness and stress form seven-day means. Injury/Performance.
injuries	injury_occurred is the daily outcome log and source of previous injury count. injury_type and days_missed are stored/displayed, not model features.
matches	rating is the Performance target and prior-rating feature; minutes/goals/assists form recent sums. shots, passes_completed, key_passes are stored/displayed, not ML features.
health + tests	Nine required daily health values form means/slopes/changes; six optional values provide recent latest values; eleven lab values provide latest/change/recency. Health ML and trends.
health_events	health_event is the separate daily target log for Health ML; it is not inferred from measured health fields.

MISSING / INVALID / DUPLICATE

Missing required column blocks validation; missing required cell rejects that row. Missing optional cell becomes NULL. Invalid date, Boolean, range or cross-field rule rejects row. Unknown player_id rejects row. Duplicate player ID or player/date rejects row. Default import is all-or-nothing; "Import valid rows only" can save valid rows while reporting errors.

04 Upload to ML-ready row

The same validation and transaction path handles CSV and manual entry.

STEP	ACTUAL BEHAVIOR
1 Input	Choose template; read CSV as strings; auto-detect a dataset when possible; user confirms dataset and column mapping. Unmapped columns are ignored.
2 Schema	Each required mapped column must exist. One input column cannot map to two fields. Required empty cells are errors.
3 Type/range/date	Dates must be YYYY-MM-DD and not future; numeric values finite and in field bounds; designated integers whole; Booleans 0/1, true/false, yes/no.
4 Cross-field	Known player_id; training sprint/high-speed each $\leq \text{distance} \times 1000$; match goals $\leq$ shots; health systolic $>$ diastolic if both provided; unique player or player/date.
5 Save	Preview valid rows and issues. Confirm import; default blocks all rows when any issue exists. Optional valid-only mode inserts accepted rows in a nested transaction and commits.
6 Retrieve	read_dataset executes SQLAlchemy select for the chosen table and builds a Pandas frame; History/HealthHistory group by player.
7 Engineer	A snapshot at point t uses dates strictly before t. Training/recovery/health use prior seven calendar days; model-specific features are assembled.
8 Model ready	The resulting feature row is passed through the fitted imputer/scaler/encoder and then the selected estimator.

EXACT DATA PATH

RAW CSV > mapped fields > normalized valid rows > SQLite table > Pandas retrieval > prior-date snapshot > ML feature vector

Missing data at prediction time

Injury and Performance omit a player unless workload_7d and sleep_hours_7d_mean exist; Performance also requires recent_rating_mean. Health omits a player with zero health observations in the prior seven days. Other missing numeric features pass to a median imputer, with missing indicators; optional medical values can remain absent.

05 Feature engineering formulas

t = prediction date; W7 = [t-7 days,t); W28 = [t-28 days,t). All formulas use prior records.

FEATURE / RAW INPUT	EXACT FORMULA; EXAMPLE; USED BY
training_load; duration_min,rpe	duration_min * rpe. Example 60*7=420. Intermediate session load for workload sums.
workload_7d / 28d	Sum of training_load in W7 / W28. Example loads 420+300=720 / 4 weeks of 720=2880. Injury + Performance.
workload_change_pct	(workload_7d/(workload_28d/4)-1)*100; example 720 vs 2880/4 = 0%. NaN without full 28-day history or positive baseline. Injury + Performance.
sprint_load_7d; sprint_distance	Sum sprint_distance in W7; example 100+80=180 m. Injury + Performance.
high_speed_load_7d; high_speed_distance	Sum high_speed_distance in W7; example 300+250=550 m. Injury + Performance.
six recovery 7d means	Mean of W7 values for sleep_hours, sleep_quality, hrv, resting_hr, soreness, stress; e.g. HRV (50+60)/2=55. Injury + Performance.
previous_injuries	Count prior injury_occurred=True where injury_date<t; e.g. 2 previous onsets -> 2. Injury only.
recent_rating_mean	Mean ratings in latest five prior matches; e.g. (7+8+6)/3=7. Performance only.
rating_trend	Slope of least-squares line through prior rating order x=0..n-1; e.g. 6,7,8 -> +1 per match. Performance.
rating_consistency	Population standard deviation of last five prior ratings; e.g. 7,7,7 -> 0. Performance.
recent_minutes/goals/assists	Sum each field across latest five earlier matches; e.g. 80+90=170 minutes. Performance.
age / position	Profile values copied unchanged; age numeric, position one-hot encoded. Injury + Performance.

Health feature families - exact expansion

For each of 9 required health fields f, create f_7d_mean = mean(W7), f_trend = least-squares slope against calendar days in W7, and f_change = last(W7)-first(W7). Example HRV 50,60 over two days -> mean 55, trend +10/day, change +10. All 27 enter Health ML.

For each of 6 optional health fields, latest_f = last non-null W7 value. For each of 11 medical fields, latest_f = last value before t; change_f = last-previous; days_since_f = t-last test date. Example ferritin 30 then 35 with last test 4 days ago -> latest 35, change +5, days_since 4. Add health_observations_7d = count W7 rows and days_since_health = t-last health date. Total Health ML inputs: 68. Missing history yields NaN and is imputed.

06 Injury prediction: every step

One training row follows a recorded training day; forecast starts the next midnight.

STEP	WHAT HAPPENS
1 Input	players + training + recovery + injuries are retrieved; matches are read for snapshot but not in Injury feature list.
2 Validation/storage	Import rules on pages 3-4 apply; tables players, training, recovery, injuries hold the raw values.
3 Features	age, position, 5 training values, 6 recovery means, previous_injuries (14 inputs). All dated features precede t.
4 Target	t = training date + 1; y=1 if any injury_occurred=True in [t,t+7 days). y=0 only if all seven dates have explicit injury outcome rows and none is True. A same-day onset excludes the example.
5 Readiness	At least 40 usable windows, 5 positive windows and 5 recorded onsets; both labels; required feature availability; viable chronological partitions.
6 Split/leakage	Distinct dates split at 60% and 80%; purge rows whose label_end crosses next boundary. >=5 rows each. Same-day feature records excluded. Players can appear across periods.
7 Preprocess	Numeric median imputation + missing indicators + StandardScaler; position OneHotEncoder(handle_unknown=ignore). Fitted on training partition only during comparison.
8 Candidates	Logistic Regression (balanced linear log-odds); Random Forest (100 balanced trees, min leaf 2); Gradient Boosting (sequential trees). Each receives the same preprocessed features and predicts class/probability.
9 Select/test/save	Validation score = 0.7*recall + 0.3*F1; highest wins, ties follow listed order. Refit winner on train+validation, evaluate untouched test, save joblib + JSON.
10 Predict/UI	predict_proba(X)[:,1] = estimated onset probability. Low p<0.4; Medium 0.4<=p<0.7; High p>=0.7. ML Predictions/Dashboard/Players show p and band.

IMPORTANT INTERPRETATION

The displayed percentage is the fitted classifier output, not a rule-based injury score. It is uncalibrated research output. A missing outcome day is unknown, never assumed injury-free.

07 Performance prediction: every step

The supervised target is an actual match rating; prediction estimates a future rating.

STEP	WHAT HAPPENS
1-3 Input > features	Read players, prior matches, training and recovery; validate/store first. At match date t , use last 5 earlier matches, W7/W28 training and W7 recovery. Features: age, position, 6 match-history, 5 training, 6 recovery = 19.
4 Target	y = actual rating on match at t (0-10 input scale). First match per player is skipped because no earlier match exists.
5 Readiness	≥ 35 usable rows, match history, required feature coverage, ≥ 10 distinct prediction dates, ≥ 5 rows after purge in each partition, at least two training target values.
6 Split/preprocess	Same date-based 60/20/20 and boundary purge as Injury; numerical median + missing flags + scaling; one-hot position.
7 Candidates	Linear Regression fits linear coefficients. Random Forest Regressor averages 100 tree outputs (min leaf 2). Gradient Boosting Regressor adds trees to correct residuals. All output a continuous rating estimate.
8 Select/save	Choose lowest validation MAE (equivalent to max -MAE); ties use candidate order. Refit train+validation, evaluate test once, save joblib + JSON.
9 Predict/UI	For eligible players, pipeline.predict produces predicted_next_rating. It is a numerical estimate of the next match rating, not a known result. The code does not clamp it to 0-10.

Match-history calculations - a tiny example

INPUT	FORMULA / OUTPUT	REASON
ratings 6,7,8	mean=7; trend slope=+1; population SD~0.82	level, direction, variability
minutes 70,80,90	sum=240	recent playing time
goals 0,1,0; assists 1,0,1	sum goals=1; assists=2	recent contributions

Regression metrics used to compare and report

MAE = $\text{mean}(|y - \hat{y}|)$; errors 1,2 -> MAE 1.5, lower is better. RMSE = $\sqrt{\text{mean}((y - \hat{y})^2)}$; errors 1,2 -> $\sqrt{2.5}=1.58$, lower is better. R2 = $1 - \text{sum squared errors} / \text{sum}((y - \text{mean } y)^2)$, higher is better; can be negative. Model selection uses MAE only; the other two appear in evaluation.

08 Health monitoring: every step

Health measurements are inputs; health_events is the independent outcome log.

STEP	ACTUAL HEALTH PIPELINE
1 Input	health.csv: 9 required daily fields + 6 optional vitals/body fields; medical_tests.csv: test_date + 11 optional labs; health_events.csv: daily label observations.
2 Validate/store	Same import checks; accepted rows enter health_records, medical_tests, health_events. A medical panel may have no measured values and still be accepted with warning.
3 Features	For each required field: W7 mean, calendar-day slope and last-first change (27). Six optional latest W7 values. Eleven lab latest/change/days-since triples (33). Count W7 health rows and days since last = 68 inputs.
4 Target	For each health observation day d, t=d+1. y=1 if any health_event=True in [t,t+7). y=0 only if all seven follow-up dates are explicitly logged and none is True; otherwise skip.
5 Readiness	>=40 labeled windows, >=5 documented events, both classes, valid 60/20/20 chronological partitions with >=5 rows each and both classes in train/validation.
6 Preprocess	Numeric median imputation (keep all-empty fields) + missing indicators, then StandardScaler. Optional or absent lab features are not claimed to be observed.
7 Train/select	Logistic Regression, Random Forest Classifier (100 trees, balanced, min leaf 2), Gradient Boosting Classifier. Maximize 0.7*validation recall + 0.3*validation F1; candidate order breaks ties.
8 Save/test	Refit selected model on train+validation; evaluate held-out test; save health-specific joblib + JSON and reference medians.
9 Predict/UI	At least one health row in W7 required. predict_proba[:,1] -> LOW <0.4, MEDIUM 0.4-<0.7, HIGH >=0.7. Health tab shows band, percent, model metrics and local one-feature substitutions.

The 9 required measurements each create 3 features

RAW FIELD	MEAN / TREND / CHANGE FEATURE NAMES
resting_hr, hrv	resting_hr_*; hrv_*
sleep_hours, sleep_quality	sleep_hours_*; sleep_quality_*
soreness, fatigue, stress	soreness_*; fatigue_*; stress_*
energy_level, hydration_status	energy_level_*; hydration_status_*

For every * above, suffixes are 7d_mean, trend, change. W7 mean = arithmetic mean; trend = least-squares slope per calendar day (NaN with <2 dates); change = latest minus earliest W7 reading (NaN with <2). All 27 are inputs to each Health candidate, along with all optional/latest, lab and recency features.

FEATURE EXAMPLE

HRV 50 then 60 across two calendar days -> hrv_7d_mean=55, hrv_trend=+10/day, hrv_change=+10. Ferritin 30 then 35, last tested 4 days before t -> latest=35, change=+5, days_since=4. Each value enters each Health candidate.

09 Health trends and clustering

These two Health tabs serve different purposes and have no historical target labels.

Health Trends - statistical analysis, no fitted supervised model

INPUT / ACTION	FORMULA / OUTPUT / PURPOSE
Player + field	Read observed health or medical-test values in date order; absent fields show no chart.
Rolling chart	Time-based rolling mean over 7 days for health or 90 days for labs; min 1 value. Smooths observed variation.
Recent value	Last observed value; displayed in summary.
Change percent	$(\text{last}/\text{previous} - 1) * 100$, only when ≥ 2 values and previous $\neq 0$; displayed as change_pct.
Direction	Least-squares slope on last 7 observed values vs calendar days: $> 1e-8$ Rising, $< -1e-8$ Declining, else Stable; < 2 values $\rightarrow$ Insufficient history.
UI	Heart, Sleep, Recovery, Hydration, Body, Vitals, Medical tests expanders; summary table and line chart.

Health Clustering - unsupervised K-Means, no given answer

STEP	EXACT BEHAVIOR
Select records	At selected profile date, include players with ≥ 1 health row in W7; require at least k players.
Candidate features	Nine 7-day means for required health fields plus latest hemoglobin, ferritin, vitamin_d, crp.
Keep / fill / scale	Keep feature when $\geq 50\%$ nonmissing and > 1 unique value; median-impute gaps, StandardScaler.
K-Means	User chooses $k=2,3,4,5$ (default 3). Require $\geq k$ distinct scaled profiles. Fit <code>KMeans(random_state=42,n_init=10)</code> ; display cluster numbers 1..k.
Interpret	Profile = per-cluster raw-feature mean. Compare to roster mean / roster SD; report three largest absolute differences as higher/lower average.
Visualize	PCA projects scaled features to up to 2 components ($PC2=0$ if one feature); Plotly scatter colors points by cluster. PCA is for display, not K-Means input.

NO CLUSTERING SCORE

The code displays assignments, group sizes implicitly through rows, profiles and a PCA chart. It does not calculate a silhouette score.

10 What supervised ML means here

Past inputs and known outcomes train three models. New inputs produce estimates.

MODULE	HISTORICAL INPUT	KNOWN TARGET	TYPE	FINAL OUTPUT
Injury	Earlier training/recovery/profile/injury count	New onset within next 7 dates	Supervised classification	Probability + risk band
Performance	Earlier matches/training/recovery/profile	Actual rating on match date	Supervised regression	Estimated next rating
Health	Earlier health/test measurements	Event within next 7 dates	Supervised classification	Probability + risk band
Health Trends	Observed series	None	Statistics + charts	Mean/change/direction
Health Clustering	Recent health/lab profile	None	Unsupervised K-Means	Cluster/group profile

TRAINING

PAST INPUTS + DOCUMENTED OUTCOME > preprocess on training data > fit each candidate > compare validation results > refit winner > test

PREDICTION

NEW PRIOR-DATE INPUTS > saved preprocessing + selected model > probability or rating estimate > risk band when classification > UI

What a label does - and does not - mean

An Injury or Health positive training row means at least one documented new onset occurred in the following seven calendar dates. A negative row requires explicit zero observations on every one of those dates. A missing date provides no answer. Performance uses the actual rating from the match at that date. Health measurements themselves do not create health_event labels.

Why the time split matters

A model is selected on later validation dates and reported on still later test dates. Feature snapshots exclude same-day and future values. Rows whose outcome window crosses a partition boundary are removed from the earlier partition. This limits temporal leakage; the same players can still occur in all periods, so test results are about later records, not unseen players.

11 Evaluation, cutoffs and validation ranges

Metrics below are computed on model outputs; input bounds are software checks, not clinical limits.

METRIC	FORMULA / RANGE / DIRECTION / PROJECT MEANING
Accuracy	$(TP+TN)/N$; 0..1 higher. Overall correct class share.
Precision	$TP/(TP+FP)$; 0..1 higher. Among predicted onsets, share documented.
Recall	$TP/(TP+FN)$; 0..1 higher. Share of documented onsets caught.
F1	$2*precision*recall/(precision+recall)$; 0..1 higher. Balances precision/recall.
ROC-AUC	Area under TPR vs FPR across cutoffs; 0..1 higher. Probability ranking (None if test has one class).
Confusion matrix	$[[TN,FP],[FN,TP]]$; counts ≥ 0 . Shows exact class error types.
MAE / RMSE	$mean e / \sqrt{mean e^2}$; ≥ 0 lower. Rating error size; RMSE weights large misses more.
R2	$1 - SSE/SST$; ≤ 1 , higher. Fit vs predicting mean rating; may be negative; None for one actual value.
Clustering	No silhouette metric in code. Cluster IDs, member rows and observed cluster means are shown.

DECISION CUTOFFS

Injury: Low $p < 0.40$, Medium $0.40 \leq p < 0.70$, High $p \geq 0.70$. Health: LOW / MEDIUM / HIGH at the same values. Selection: classification $\max(0.7*recall + 0.3*F1)$; Performance min validation MAE.

Actual upload ranges (inclusive)

RPE, sleep_quality, soreness, stress, fatigue, energy_level, hydration_status, rating: 0-10. sleep_hours: 0-24; HRV: 1-300; resting_hr: 20-220. age: 10-70; height: 100-250; weight: 25-200; duration_min: 1-360; distance km: 0-60; sprint and high-speed distance m: 0-60000.

Match: minutes 1-130; goals/assists 0-30; shots/key_passes 0-100; passes_completed 0-300. days_missed 0-3650. Body fat and oxygen saturation 0-100; systolic 40-300; diastolic 20-200; temperature 25-45. Medical fields 0-100000, except hematocrit 0-100. Dates cannot be future.

12 Complete UI to backend walkthrough

Each row follows a user action to its query, calculation and visible result.

PAGE / ACTION	FUNCTION > QUERY > CALCULATION / MODEL > DISPLAY
Dashboard / open	dataset_counts + readiness read all tables; prediction_tables loads saved Injury/Performance artifacts, rebuilds features and predicts; health summary calls Health artifact. Counts, readiness, predictions, elevated-risk count.
Data / Import	upload > pd.read_csv > mapping > validate_input > import_records > INSERT via SQLAlchemy. Preview, errors, saved-row notice; Records tab SELECTs/export CSV.
Data / Save record	Manual form > same validation/import transaction. Invalid values shown; successful save reruns page.
Players / choose	read_dataset for roster, training, recovery, matches, injuries; calculate training_load; History.snapshot; prediction_tables. History tables, load/rating charts, features and model predictions.
ML Predictions / date	load_artifact > History.prediction_frame > predict_proba or predict. Probability/band and estimated rating, plus global feature list; unavailable artifact/history shows message.
Model Analysis / Train	readiness > History.training_frame > temporal_split > preprocess + three candidate estimators > validation selection > refit > test > joblib/JSON. Comparisons, metrics, matrix/importance.
Health / Train	HealthHistory.training_frame + assess_health > three classifiers > validation selection > held-out test > health artifact. Readiness and evaluation.
Health / Predict	Select player/date > HealthHistory.snapshot > saved pipeline.predict_proba > band + explain_prediction reference substitution. Risk, percent, feature differences.
Health / Trends	Read selected player field > trend_analysis; no model. Summary and line chart.
Health / Cluster	HealthHistory.prediction_frame > feature filter/impute/scale > KMeans + PCA; assignments, profiles, summaries and scatter.

ONE MASTER DIAGRAM

RAW DATA > VALIDATION > DATABASE > FEATURE FORMULAS > ML FEATURES > MODEL / ANALYTICS > PREDICTION OR GROUP > RISK BAND / SCORE > EVALUATION > STREAMLIT UI

Viva checklist

1. Name the eight CSVs and three label sources. 2. Explain why absent outcome dates are unknown. 3. Derive training load, workload change, rating trend and one Health feature. 4. State 60/20/20 chronological split and purge. 5. Name all candidate models and selection formulas. 6. Distinguish class probability, rating regression, trends and K-Means. 7. State 0.40/0.70 cutoffs and UI destinations.